

DLNA Gateway

Refactoring & Test Plan

Version 1.0 — 15 April 2026

Project: ~/dlna-gateway/ on Mac Mini M4 (Hoorn)

This document defines the refactoring plan for the DLNA Gateway, a personal music gateway bridging AssetUPnP, IINA, Chromecast, and UPnP renderers. The goal is to break the monolithic codebase into maintainable modules, add regression tests, and prevent the cascading failures experienced during the April 2026 sessions.

Current state

- 7 Python modules, ~5400 lines total
- 1300 lines of HTML/CSS/JS embedded inside `dlna_gateway.py` as a Python string
- `dlna_server.py`: 1100-line monolith with 30+ API endpoints in one class
- SQLite with no connection pooling — vulnerable to 'database locked' under load
- No tests — regressions discovered only by manual testing
- Under git as of April 2026

Guiding principles

- Separation of concerns — each file does one thing
- Static frontend — HTML/CSS/JS in their own files, served as static assets
- Functional style — module-level functions, simple data classes, no class hierarchies
- Test before ship — automated checks for API endpoints, DB integrity, UI serving
- Git discipline — one commit per logical change, test before merge

Phase 1 — Extract the frontend

Remove all HTML, CSS, and JavaScript from `dlna_gateway.py` and place them in separate static files. This is the highest-impact change — it eliminates the root cause of most regressions (editing JS inside a Python string).

Files created

- `static/index.html` — HTML shell (~100 lines)
- `static/app.css` — all CSS including mobile responsive (~300 lines)
- `static/app.js` — all JavaScript (~900 lines), merged from current + March 29 browse system
- `static/sw.js` — service worker (moved from Python string constant)

Files modified

- `dlna_gateway.py` — shrinks from ~1700 to ~200 lines. Only `main()`, startup, thread wiring.
- `dlna_server.py` — add static file serving: `GET /` serves `index.html`, `GET /static/*` serves CSS/JS

Merge scope

The March 29 browse system (letter bar, alphabet picker, drill-down navigation with smart back, genre browsing, metadata edit modal, pagination) is merged into app.js during this phase. The current Chromecast, shuffle, PWA, HTTPS, and visibility polling features are preserved.

Tests — Phase 1

T1.1 — Static file serving

GET / returns HTML containing 'DLNA Gateway'. GET /static/app.js returns JS. GET /static/app.css returns CSS. Correct Content-Type headers.

T1.2 — Service worker

GET /sw.js returns JavaScript with Content-Type application/javascript and no-cache headers.

T1.3 — PWA manifest

GET /manifest.json returns valid JSON with name, start_url, display, icons.

T1.4 — Icons

GET /icon-192.png and /icon-512.png return valid PNG (first 8 bytes = PNG signature).

T1.5 — API endpoints survive

All 32 API endpoints return valid JSON (not 404). Spot-check: /api/servers, /api/artists, /api/playlists.

T1.6 — Mobile responsive

index.html contains viewport meta tag, app.css contains @media (max-width:768px) rules.

Phase 2 — Split the server

Break dlina_server.py into focused API modules. The HTTP handler becomes a thin router that dispatches to module functions.

Files created

- **api_browse.py** — browse, artists, albums, genres, search, browse_letter, artist_albums (~150 lines)
- **api_playback.py** — play, control, state, render_queue, cast_queue, stream proxy (~200 lines)
- **api_playlists.py** — playlists CRUD, favourites, add/remove/create/delete (~100 lines)
- **api_upnp.py** — UPnP ContentDirectory SOAP gateway for Naim Uniti (~150 lines)

Files modified

- **dlina_server.py** — shrinks from ~1100 to ~300 lines. HTTP server + router + static serving.

Tests — Phase 2

T2.1 — API module imports

Each api_*.py module imports cleanly with no circular dependencies.

T2.2 — Endpoint coverage

Automated test hits all 32 endpoints, verifies HTTP 200 or 400 (not 500/404).

T2.3 — Browse letter

/api/browse_letter?mode=artists&letter=S returns items with artists starting with S.

T2.4 — Edit track

POST /api/edit_track with valid URL updates the track, GET confirms change.

T2.5 — Cast devices

/api/cast_devices returns a list (empty is OK if no Chromecast on test network).

Phase 3 — Database connection pool

Replace per-request `sqlite3.connect()` with a proper connection pool. This fixes 'database is locked' errors under concurrent thread access and prevents corruption during crash-recovery.

Files created

- **db_pool.py** — SQLite connection pool: WAL mode, `busy_timeout=5000ms`, max 4 connections, context manager interface, proper cleanup on shutdown

Files modified

- **dlina_library.py** — replace `self._connect()` calls with `pool.get_connection()` context manager

Configuration

- WAL mode enabled on pool creation (already set, but enforced)
- `busy_timeout=5000` — wait up to 5s for a lock instead of failing immediately
- Max 4 connections — enough for the threaded server, prevents FD exhaustion

Tests — Phase 3

T3.1 — Concurrent access

10 threads simultaneously call `DB.search()` and `DB.track_count()` — no 'database locked' errors.

T3.2 — WAL mode verified

`PRAGMA journal_mode` returns 'wal' on every connection from the pool.

T3.3 — Connection cleanup

After `pool.shutdown()`, all connections are closed, no leaked file descriptors.

T3.4 — Integrity under load

Run index rebuild while simultaneously serving browse/search requests. `PRAGMA integrity_check` passes after.

Phase 4 — Server heartbeat

Add a background heartbeat that pings AssetUPnP every 30 seconds, eliminating the 'Server offline — reconnecting' flicker.

Files modified

- **dlina_discovery.py** — add `heartbeat_thread()` that fetches the server's `DeviceDescription.xml` every 30s and updates `last_seen` on success

Tests — Phase 4

T4.1 — Heartbeat updates last_seen

After heartbeat fires, `server.last_seen` is within 35 seconds of current time.

T4.2 — Offline detection

If AssetUPnP is stopped, server shows offline within 60 seconds.

T4.3 — No false offline

With AssetUPnP running, `/api/servers` never returns `online:false` for more than one poll cycle.

Phase 5 — Frontend polish

With the JS in its own file (Phase 1 complete), restore and enhance the full browse experience from the March 29 version.

Features

- **Letter bar** — alphabet picker (A-Z + # for numbers) at top of Artists/Albums/Tracks views
- **Smart drill-down** — Genre → Album → Tracks with back navigation preserving context

- **Playlist dropdown scroll** — max-height: 200px with overflow-y: auto
- **Metadata edit modal** — edit artist/album/title/genre per track via /api/edit_track
- **Shuffle button** — toggle in now-playing panel (currently only works via Radio button)
- **Browse pagination** — 100 items per page with prev/next for large letter groups

Tests — Phase 5

T5.1 — Letter bar rendering

app.js contains LETTERS array with 28 entries (# 0 A-Z). buildLetterBar() produces buttons.

T5.2 — Drill-down stack

browseNavStack correctly pushes/pops levels. drillBack() returns to previous context.

T5.3 — Various Artists handling

playAlbumFromDB('Various Artists', album) passes empty artist to API.

T5.4 — Shuffle state

Toggling shuffle on/off correctly reorders/restores browserQueue.

Phase ∞ — Test framework

A lightweight test runner that can be executed before every deploy. No external test framework needed — pure Python + urllib.

Implementation

- **tests/test_api.py** — HTTP-level tests against a running gateway instance
- **tests/test_db.py** — SQLite pool tests (concurrent access, WAL mode, integrity)
- **tests/test_static.py** — static file serving, content types, PWA manifest validation
- **tests/run_all.py** — runner script, exits non-zero on any failure

Usage

```
python tests/run_all.py http://localhost:8765
```

Run before every deploy. Takes ~5 seconds. Red/green output. Integrates with git pre-push hook if desired.

Test matrix summary

Phase	Tests	What they cover
1 — Frontend	T1.1–T1.6	Static serving, PWA, endpoints survive extraction
2 — Server split	T2.1–T2.5	Module imports, all 32 endpoints, browse_letter, edit_track
3 — DB pool	T3.1–T3.4	Concurrent access, WAL mode, cleanup, integrity under load
4 — Heartbeat	T4.1–T4.3	last_seen updates, offline detection, no false positives
5 — Frontend polish	T5.1–T5.4	Letter bar, drill-down stack, Various Artists, shuffle

Execution order and rationale

Phase 1 first — eliminates the root cause of regressions (JS in Python strings). Every other phase benefits from this being done. **Phase 3 second** — prevents data corruption, which has cost the most recovery time.

Phase 2 third — server split is nice but lower risk; the server rarely breaks. **Phase 4** — trivial, can be done anytime. **Phase 5** — depends on Phase 1.

Each phase ends with: tests pass, git commit, deploy to Mac Mini, verify in browser, commit again if any hotfix needed.